# Entelechy Continuum Engine

*Corrected kernels, catalog, and build cut. Version 0.1.0.*

This handbook compiles the design documents in `docs/`. Executable truth is `ece/*.py` and `tests/`. The original catalog remains lineage reference; formulas here supersede it.

# Entelechy Continuum Engine — Overview

ECE is a hybrid Eulerian–Lagrangian artificial-life runtime. It does not collapse every rule family into one equation. It runs them as plugins on a shared state:

- Eulerian fields (Lenia, SmoothLife, Field Life, MaCE, RD)
- Lagrangian particles (Mohr Particle Life, Clusters, PPS, Boids)
- Particle-in-Cell coupling (deposit / sample) for hybrids

This repository ships corrected CPU reference kernels, Tk and native WebGPU previews, and optional CUDA/WebGPU compute backends with oracle tests. Fully device-resident hybrid execution and co-local AI remain deferred.

## Two axes that must not be mixed

**Lagrangian vs Eulerian** — points moving through space vs density on a grid.

**Conserving vs open-growth** — mass invariant (MaCE, Flow-Lenia, Field Life) vs source/sink (classic Lenia, GoL, Mohr).

Asymmetric species matrices violate momentum conservation on purpose. That is how chase rings and organelles appear.

## Source of truth

| Layer | File |
|---|---|
| Spec deltas vs the original catalog | `CORRECTIONS.md` |
| Executable math | `ece/*.py` |
| GPU fragments | `shaders/*.wgsl` |
| Invariants | `tests/` |
| Family catalog | `docs/02-rules-catalog.md` |
| Build cut | `docs/09-roadmap.md` |

## Default first run (M1)

2D Mohr Particle Life, torus world, 6 species, live matrix, spatial hash, exponential friction. No 3D, no VLM, no FFT.

# Lineage

Tom **Mohr** of particle-life.com, not “Tom Moore.” Clusters (Jeffrey Ventrella) came first. Mohr simplified the per-pair piecewise fields into one global tent plus one matrix. Field Life moves that matrix onto density fields and adds MaCE so mass does not evaporate. Lenia is a different line: Conway’s Life made continuous.

```
GoL (B3/S23)
  ■■ Life-like CA (B/S notation)
  ■■ Larger-than-Life / Bugs
  ■■ SmoothLife (disk + annulus + birth/survive intervals)
  ■    ■■ SmootherLife (Gaussian kernels)
  ■■ Lenia (radial kernel + growth bell)
       ■■ multi-channel / multi-kernel
       ■■ Glaberish (genesis + persistence)
       ■■ Flow-Lenia (mass-conserving flow)
       ■■ MaCE-Lenia
       ■■ Particle Lenia (energy on dots)

Clusters (Ventrella): per-pair (r1,r2,f1,f2) + asymmetric horizons
  ■■ Particle Life (Mohr): one tent + one matrix
  ■    ■■ hunar4321 / najarro.science / par-particle-life
  ■    ■■ Field Life = matrix + kernel + MaCE
  ■■ Clusters X / Atomic Clusters (ciphrd)

PPS (Schmickl): Δϕ = α + β N sign(R − L)
Boids (Reynolds): separate / align / cohere
NCA: learned local net
Reaction-diffusion: Gray–Scott, FitzHugh–Nagumo
```

## What each ancestor contributes

| Ancestor | Keep |
|---|---|
| Clusters | Asymmetric matrix, optional per-pair radii later |
| Mohr | Global tent, one `r_max`, one `beta`, spatial bins |
| Field Life | Convolution + matrix + conservative transport |
| Lenia | Kernel + unimodal growth, optional multi-channel |
| MaCE | Softmax-9 / softmax-27 mass routing |
| PPS / Boids | Heading agents; same bins, different force pass |

Particle Lenia is the Lagrangian reading of Lenia’s energy, not a Mohr variant. ECE implements its overdamped energy-gradient rule as a separate single-species periodic CPU reference; see the catalog for its equation and torus adaptation.

# Rules catalog

State, interaction, and conservation. Implementation parameters live in `configs/` and `schemas/sim.schema.json`.

## Family table

| Family | State | Interaction | Conserves mass? |
|---|---|---|---|
| Life-like CA | discrete cells | neighbor count | no |
| SmoothLife / Lenia | continuous field | convolution + growth | no (unless Flow/MaCE) |
| Clusters / Particle Life | colored points | pairwise force matrix | no |
| Field Life | colored density fields | kernel + matrix + transport | yes, per color |
| Particle Lenia | points | energy gradient | no, unless added |
| PPS / Boids | heading agents | local geometry | no |

## Side-by-side

| System | Substrate | State | Topology | Primary parameters | Conserves | Cost per step |
|---|---|---|---|---|---|---|
| GoL / Life-like | discrete grid | {0,1} | Moore 8 | B / S counts | no | O(N) |
| SmoothLife | field | [0,1] | disk + annulus | radii, birth/survive intervals, α | no | O(N log N) FFT |
| Lenia | field | [0,1] | radial kernel | shells β, (μ,σ), dt | no | O(N log N) |
| Flow-Lenia / MaCE-Lenia | field | [0,1] | kernel + flux | Lenia + transport β | **yes** | O(N log N)+O(N) |
| Clusters | particles | x,v,c | pairwise, directed | per-pair (r1,r2,f1,f2), γ, delay | no | O(M k) bins |
| Particle Life (Mohr) | particles | x,v,c | pairwise ≤ r_max | matrix A, β, λ | no | O(M k) hash |
| Field Life | density grid | ρ_c ≥ 0 | conv + matrix + MaCE | K, M, λ_crowd, β_tr | **yes** | O(K N log N) |
| Particle Lenia | particles | x | radial shell KDE + energy gradient | μ_K, σ_K, μ_G, σ_G, c_rep | no | O(M²) reference |
| PPS | agents | x, ϕ | left/right count | r, α, β, v | no | O(M k) |
| Boids / Swarm Chemistry | agents | x, v | neighborhood means | w_sep, w_align, w_coh | no | O(M k) |

## Update rules (compressed)

### Conway / Life-like

Birth on count set B, survive on S. Classic B3/S23.

### SmoothLife

Inner disk fill `m`, outer annulus fill `n`, logistic mix of birth interval `[b1,b2]` and survival `[d1,d2]`. Famous glider: birth [0.278, 0.365], survive [0.267, 0.445], α_n ≈ 0.028, α_m ≈ 0.147.

### Lenia

`U = K * A`, `G(u) = 2 exp(-(u-μ)² / 2σ²) - 1`, `A ← clip(A + dt G(U), 0, 1)`. Orbium: ring near r=0.5, μ≈0.15, σ≈0.015.

### Mohr Particle Life (corrected)

See `docs/03-mohr.md`. Tent + matrix. No extra `r_max` on the unit vector.

### Field Life / MaCE (corrected)

See `docs/04-mace.md`. Two-pass softmax transport on a torus.

### Particle Lenia

The CPU reference follows the author energy-gradient rule. For particle positions `p_i`, define the normalized radial-shell density `U(x) = Σ_i K(||x-p_i||)`, where `K(r) = w_K exp(-((r-μ_K)/σ_K)^2)`. Growth is `G(u) = exp(-((u-μ_G)/σ_G)^2)`. Short-range repulsion has potential `R(x) = c_rep/2 Σ_{j:p_j != x} max(1-||x-p_j||,0)^2`; local energy is `E(x) = R(x) - G(U(x))`, and overdamped motion is `dp_i/dt = -∇E(p_i)` with other particles held fixed.

ECE adapts the continuous reference to its periodic 2D world using minimum-image distances and wrapped positions. The separate `particle_lenia.toml` config uses a 16×16 world so the shell radius is not collapsed by the unit-torus scale. This rule has one species and no Mohr matrix, inertial velocity, or PIC field state.

The equations follow the authors' [Particle Lenia description](https://google-research.github.io/self-organising-systems/particle-lenia/) and [reference notebook](https://github.com/google-research/self-organising-systems/blob/master/notebooks/particle_lenia.ipynb); periodic boundaries and ECE config integration are local adaptations.

### PPS

`Δϕ = α + β N sign(R-L)`, constant speed. Default life-like: r=5, α=180°, β=17°, v=0.67.

### Boids

Separation, alignment, cohesion. Swarm Chemistry assigns different weight triples per species — same “matrix of types” idea, geometric steering instead of a radial tent.

## DAG passes per rule

| Rule | Passes |
|---|---|
| Mohr / Clusters | Forces, Integrate |
| Field Life | Convolve, TransportDenom, TransportGather |
| Lenia / SmoothLife | Convolve, Growth |
| Flow-Lenia / MaCE-Lenia | Convolve, Growth, Transport* |
| Particle Lenia | Shell KDE, EnergyGradient, Integrate |

| Rule | Passes |
|---|---|
| PPS / Boids | Steer, Integrate |
| Hybrid PIC | Deposit + any of the above + Sample |

# Mohr Particle Life — corrected force

Particles: position `x`, velocity `v`, species `c ∈ {0..K-1}`. One global `r_max`, one global wall `beta ∈ (0,1)`, one matrix `A[K,K] ∈ [-1,1]`, optional scalar `gain` (default 1).

## Tent

Normalize `r = ||x_j - x_i||_wrap / r_max`.

```
F(r, a) =
    r/beta - 1                              if r < beta
    a * (1 - |2r - 1 - beta| / (1-beta))    if beta ≤ r ≤ 1
    0                                       if r > 1
```

- `r < beta`: everyone repels everyone. `F` does not depend on `a`.
- Mid-range peak is at `r = (1+beta)/2` and equals `a`.
- Horizon is `r_max`. Wrapped shortest vector on the torus.

## Acceleration (the correction)

```
a_i = gain * Σ_{j≠i} F(r_ij, A[c_i,c_j]) * hat(x_j - x_i)
```

Do **not** multiply by `r_max`. That factor made peak force depend on world units and broke “same matrix, different r_max” comparisons.

`ece.mohr.mohr_accelerations` is the O($N^2$) oracle used by tests. Production uses the spatial hash in `docs/05-bins-integrate.md`.

## Integration

```
v ← exp(-λ dt) v + a dt
x ← wrap(x + v dt)
```

`λ = 0` is undamped. To recover Mohr’s `gamma**(60 dt)` feel:

```
λ = -ln(gamma) * 60
```

See `ece.integrate`.

## Asymmetry

`A` is not required to be symmetric. `A[i,j] ≠ A[j,i]` is the mechanism for orbits and chase rings. Total momentum is not a conserved quantity and must not be treated as a test invariant.

## Clusters extension (later)

Replace the single tent with per-pair `(r1, r2, f1, f2)` and optional view-radius / delay. Keep `f1` non-attractive so particles cannot collapse through the wall.

## Tests that lock this

- wall independent of matrix entry and strictly negative
- tent peak equals `a` at midpoint
- F = 0 beyond horizon

- scale world and `r_max` together → identical accelerations
- asymmetric chase pair produces net momentum

# MaCE / Field Life — two-pass torus transport

Field Life keeps Mohr's matrix, drops the dots, and moves mass on a grid so numerical friction cannot evaporate creatures.

## Sense (affinity)

Each color `c` is a field `ρ_c ≥ 0`.

```
A_c = strength * Σ_d M[c,d] (K * ρ_d) - λ_crowd ρ_c
```

Default `K` is a signed ring (negative near 0, positive on a shell) so hollow shells beat solid blobs. Convolution is periodic.

## Move (MaCE)

**Pass A — denominators**

```
Z[x] = Σ_{y ∈ N(x)} exp(β A[y])
```

**Pass B — gather**

```
ρ'[x] = Σ_{y ∈ N(x)} ρ[y] * exp(β A[x]) / Z[y]
```

`N(x)` is the 9-neighborhood (2D) or 27-neighborhood (3D), **including self**, wrapped on the torus.

Implementations evaluate each origin's neighborhood weights after subtracting that neighborhood's maximum affinity. This log-sum-exp rescaling cancels between numerator and denominator, preserving the transition exactly while preventing overflow and local denominator underflow.

## What was wrong in the catalog shader

Recomputing neighbor softmax denominators inside the gather without a stored `Z`.

`continue` on out-of-bounds instead of wrap — mass leaks at the box face.

No unit test that `Σ ρ'_c = Σ ρ_c`.

## Invariants

- Per-channel mass is bit-stable within float rounding.
- Momentum is not conserved.
- A uniform field is a fixed point when the kernel mean is 0 and crowding is applied evenly.
- `Z[x] > 0` always (`exp` is positive; neighborhood is non-empty).

## Implementation

Python: `ece.mace.mace_step`. WGSL: the executable passes, including the local-maximum rescaling, are inline in `ece/wgpu_field_life.py`. `shaders/mace_2pass.wgsl` is a shorter reference fragment without the rescaling; do

not ship it for large $\beta \cdot A$.

Multi-channel: run A and B per channel. $Z$ is computed from that channel's affinity, not shared across colors, unless you deliberately couple transport.

## 3D

Same formulas, 27 neighbors, and periodic rolls on three spatial axes. A CPU reference and CuPy/cuFFT path are available; see `tools/benchmark_cupy_field_life.py` for current target-specific volume timings.

# Spatial hash and integration

Neighbor search must be O(M k), not O(M²). Uniform bins are enough for a torus with a single `r_max`.

## Cell size

`cell_size >= r_max` lets a 3×3 (2D) or 3×3×3 (3D) walk cover the horizon. To refine bins, use `cell >= r_max / 2` and walk 5×5. Document the choice; do not mix them at runtime.

The grid needs at least `neighborhood` bins per axis. With fewer, the torus wrap maps two walk offsets onto the same bin and its particles are counted twice. `ece.bins.validate_hash_grid` enforces this for the loader and every backend.

## Four arrays, not two

| Buffer | Role |
|---|---|
| `counts` | frozen particle-per-bin snapshot |
| `offsets` | exclusive prefix sum of `counts` |
| `ranges` | `(start, end) = (offsets[b], offsets[b]+counts[b])` |
| `cursor` | scatter write head, discarded after the pass |

`sorted_indices[start:end]` are the particles in that bin.

Never exclusive-scan a counter you later `atomicAdd` as a cursor. The catalog mixed those roles.

## Algorithm

1. Clear `counts`.
2. Count: `counts[bin(x_i)] += 1`.
3. Snapshot `counts`. Exclusive scan → `offsets`.
4. Pack `ranges`.
5. Scatter into `sorted_indices` using a copy of `offsets` as cursor.
6. Force pass walks neighbor bins via `ranges`.
7. Integrate.

`ece.bins.snapshot_and_scatter` is the CPU oracle.

## Particle-in-cell exchange

`ece.pic.deposit_particles` bilinearly scatters each particle's mass into its species channel on a periodic grid. `ece.pic.sample_fields` gathers all channels back at particle positions with the same weights. The pair preserves per-species deposited mass and satisfies the deposit/sample adjoint identity. These are CPU reference operations. `ece.hybrid.run` schedules Mohr, deposit, Field Life, and sample passes. Coupling is currently one way: the field is rebuilt from the deposit each frame, and the sampled values are exported but do not yet act on particle forces. CUDA hybrid keeps particle, velocity, field, and scratch buffers on-device and uses stable CuPy sorts for hash and deposit planning. WebGPU currently transfers pass state per frame. Both PIC GPU paths reduce target-cell segments without floating-point atomics.

## Integration

Symplectic Euler + wrap:

```
decay = exp(-λ dt)
v ← decay v + a dt
x ← x + v dt
x ← x - floor(x / world) * world
```

World is a unit torus unless a config says otherwise. Positions must stay in `[0, world)`.

## Determinism note

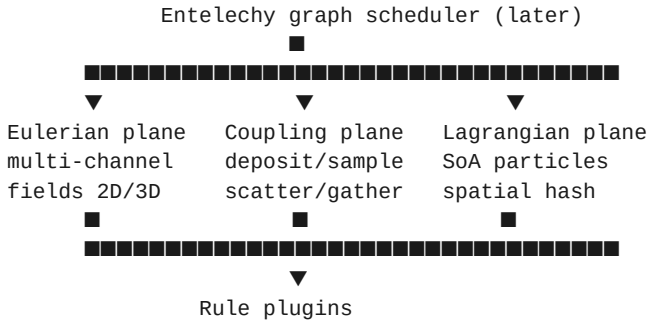
Float GPU atomics and vendor `exp` make wgpu preview non-replayable. If replay logs matter, freeze:

- particle order after scatter (already sorted by bin then arrival)
- neighborhood walk order (fixed nested dx,dy)
- a fixed-point / LUT path on CUDA later

Hash state = frame counter + positions + types + matrix. Do not hash `accel`, `Z`, or `ranges`.

# Architecture

## Planes




## Shared state (logical)

```
time, dt
euler:   [C, X, Y, (Z)] f32 ping-pong   or absent
lagrange: pos, vel, type, mass, heading  or absent
matrix:  K×K f32, optional per-pair radii
kernels: radial samples + optional FFT plan
hash:    counts, offsets, ranges, sorted_indices
```

## Frame DAG

```
Clear → Count → Scan → Scatter → PackRanges
  → Forces → Integrate
  → Deposit → Convolve → Growth
  → TransportDenom → TransportGather
  → Sample → Render
```

Compile only the passes the active rules declare.

| Rule | Uses |
|---|---|
| Mohr | hash + Forces + Integrate |
| Field Life | Convolve + MaCE pair |
| Lenia | Convolve + Growth |
| Particle Lenia | direct shell KDE + EnergyGradient + Integrate (no PIC) |
| Hybrid | union |

## Plugin surface (logical, not yet code)

```
Rule.passes() -> [Pass]
EulerianRule.convolve / update
LagrangianRule.forces / integrate
HybridRule.deposit / sample
```

## Backends

| Backend | Role |
|---|---|
| Python `ece/` | oracle, tests, small N |

| Backend | Role |
|---|---|
| wgpu / WGSL | interactive preview |
| CUDA on DGX Spark | capacity + future replay |

Do not share one float shader between preview and bit-identical logs. Preview may use f32 `exp`. Replay, if required, uses LUTs and integer state on CUDA.

## Bind-group partition (wgpu)

WebGPU cap is 8 storage buffers per stage. Group by pass family:

- Group 0 uniforms (`SimParams`)
- Group 1 particles
- Group 2 fields
- Group 3 matrix / kernel
- Group 4 hash tables

Rebind between passes. CUDA ignores this limit.

## What is implemented now

CPU Mohr, Field Life, Lenia, and Particle Lenia runners; Tk and native WebGPU previews; PIC deposit/sample reference operations; and an optional CUDA Mohr force kernel with GPU-built hash tables, CUDA Field Life passes, and deterministic CUDA PIC deposit/sample primitives. WGSL compute paths cover Mohr, Field Life, Lenia, Particle Lenia, and PIC exchange. CPU/CUDA/WebGPU hybrid schedulers export sampled fields; feeding them back into particle forces is not built yet. The native Qt/WebGPU presentation host supports Mohr particles, Particle Lenia, Field Life, and Lenia fields. CUDA Mohr preview and hybrid hash/PIC planning remain device-resident. See `docs/09-roadmap.md`.

# Determinism

BYTES_ARE_LAW applies to replay logs, not to the first interactive preview.

## Why GPU floats are not enough

Vendors fuse FMA differently, approximate `exp`/`sin` differently, and schedule reductions in different orders. Two GPUs will diverge.

## Preview vs replay

| Path | Arithmetic | Promise |
|---|---|---|
| Python oracle | IEEE f64, fixed loops | tests match on one machine |
| wgpu preview | f32 WGSL | looks right, not a log |
| Python fixed-point Mohr prototype | Q16.16 position, Q8.24 velocity, Q1.15 matrix | repeatable for identical quantized input |
| CUDA fixed-point (future) | integer kernels and fixed LUTs | cross-box replay; not implemented |

## Current prototype boundary

`ece.fixedpoint_mohr` stores persistent state as integers and uses integer distance, force, and integration operations. Friction decay is quantized using decimal arithmetic. This is a CPU replay prototype, not a CUDA backend or a cross-box certification. Serialize the integer state to replay an exact starting condition.

## Target persistent formats

| Quantity | Format |
|---|---|
| Position | Q16.16 |
| Velocity | Q8.24 |
| Matrix | Q1.15 |
| Field mass | Q16.16 |

The future CUDA port must replace transcendental functions with versioned, host-generated LUTs and keep neighborhood traversal order fixed.

## State hash

xxHash3 (or BLAKE3) over:

```
frame u64 LE
particles  N × (i32 x, i32 y, [i32 z], u32 type)
field      C × X × Y × [Z] × i32   (if present)
matrix     K² × i16
```

Derived buffers are functions of that state and are not hashed.

## Tests that exist today

Python and same-device replay invariants live in `tests/`. They lock integer state repeatability, not cross-vendor GPU execution. A reproducibility registry awaits the CUDA fixed-point port.

# DGX Spark notes

128 GB unified memory is a **capacity** win. GB10-class bandwidth is not an H100. Design for locality, mixed precision, and graphs. Do not plan interactive `1024³ × 64` channels.

## Honest first budget

| Item | Size |
|---|---|
| $256^2 \times 8$ ch × 2 buffers f32 | ~4 MB |
| $256^3 \times 8$ ch × 2 buffers f32 | ~1 GB |
| 4 M particles × 32 B | ~128 MB |
| cuFFT scratch ($256^3$) | hundreds of MB |
| 7B Q4 VLM (later) | ~5 GB |

$512^3 \times 16$ is a research config, not the default.

## What unified memory is for

- Host-side clustering / BUNCH / graph tools reading particle buffers without a PCIe staging stall.
- Dumping replay frames to NVMe while the sim runs.
- Co-locating an evaluator model later.

It does not make 3D convolution cheap.

## Stack split

| Work | Tool |
|---|---|
| Field FFT ≥ radius ~6 in 3D | CUDA / cuFFT / CuPy |
| 2D preview, UI | wgpu |
| Oracle / CI | Python `ece/` |

Dawn-on-Spark is extra risk. Use CUDA on that box for the heavy field path.

## 3D memory warning

| Grid | One f32 volume |
|---|---|
| $256^3$ | 64 MB |
| $512^3$ | 512 MB |
| $1024^3$ | 4 GB |

Double-buffer × channels × scratch multiplies fast. Cap channels and resolution independently in config.

## Bandwidth habits

- bf16 / fp16 for convolution payloads; keep MaCE mass in f32.
- Spatial hash with Morton order if the force kernel is cache-cold.

- CUDA graphs once the pass list is stable.
- Persistent kernels only after M1 is correct.

# Roadmap

Cut from the original catalog. The catalog described a company-sized surface. This is the build order that can actually finish.

This file has two parts:

**Open work** — task cards written so a coding agent (or a new contributor) can pick one up cold. Each card says why it matters, which files to touch, what "done" means, how to verify it, and what hardware the verification needs.

**Completed milestones** — what already exists, so nobody rebuilds it.

Read `AGENTS.md` first for repo conventions, invariants, and the definition of done that applies to every card.

## How to use the task cards

- Pick a card whose **Depends on** items are done and whose **Hardware** you have. If the card says **Decision needed**, stop and get the owner's answer before writing code; do not pick a default silently.
- Keep a change to one card. Do not fold in unrelated fixes; add a new card instead.
- When a card is finished, move its line to the matching milestone under **Completed milestones** as `- [x]`, with any measured numbers and the hardware they were measured on, and delete the card.
- If the work reveals new problems, add a card for each using the same fields. Never leave a known problem only in a commit message.

### Hardware tags

| Tag | Meaning |
|---|---|
| `cpu` | Fully verifiable with `numpy` + `pytest`; works in a cloud container |
| `cuda` | Needs an NVIDIA GPU with Numba CUDA (and CuPy for resident/3D paths) |
| `wgpu` | Needs a WebGPU adapter (`wgpu` package); native window also needs Qt + a display |
| `gb10` | Needs the DGX Spark itself: benchmarks and memory numbers are only meaningful there |

GPU tests skip cleanly without a device, so a green CPU run says **nothing** about GPU correctness. For `cuda`/`wgpu`/`gb10` cards without that hardware: make the change, run the CPU suite and `python3 -m py_compile ece/*.py`, and state in the PR exactly which GPU commands the owner must run before the card counts as done. Do not tick the roadmap box until those commands pass.

### Priority and order

| ID | Title | Priority | Hardware | Depends on |
|---|---|---|---|---|
| R-03 | CLI backend-availability consistency | P2 | cpu | — |
| R-04 | MaCE input-shape ambiguity | P2 | cpu | — |
| R-05 | Canonical Orbium for Lenia (M3 exit) | P1 | cpu | — |
| R-06 | State hash and `io.hash_every` | P1 | cpu | — |
| R-07 | PPS rule (CPU reference) | P2 | cpu | R-02 |
| R-08 | Two-way hybrid coupling | P1 | cpu, then cuda/wgpu | **Decision needed** |
| R-09 | Remove dead CUDA hash kernels | P2 | cuda to verify | — |
| R-10 | CuPy 3D index-overflow guard | P1 | cpu guard, cuda to verify | — |
| R-11 | WebGPU parameter block and GPU hash | P2 | wgpu | — |
| R-12 | CUDA Mohr throughput (FP64 promotion) | P1 | cuda, gb10 | — |
| R-13 | Vectorize CPU fixed-point Mohr | P3 | cpu | — |
| R-14 | CUDA fixed-point replay port | P2 | cuda | R-06 |
| R-15 | Long $512^3$ run and memory accounting | P2 | gb10 | R-10 |
| R-16 | WebGPU tests in CI on a software adapter | P3 | cpu | — |
| R-17 | Remaining config value validation | P1 | cpu | — |

CPU CI runs on every pull request (`.github/workflows/ci.yml`), so a `cpu` card's PR must be green before merge. The `cpu` cards can run in parallel; they touch different files.

## Open work

### R-03 — CLI backend-availability consistency

- **Why:** in `ece/__main__.py`, `--cuda`/`--wgpu` for Particle Lenia skip the `cuda_available()`/`webgpu_available()` checks every other rule has, so users get a raw exception instead of a clean error. Some `parser.error` messages also omit Particle Lenia from the supported list.
- **Files:** `ece/__main__.py`, `tests/test_particle_lenia.py`.
- **Done when:** every backend flag on every rule gives a `parser.error` (exit code 2) naming the missing extra when the device is absent, and the messages list all supported rules.
- **Verify:** CPU tests that run the CLI with `--cuda` and `--wgpu` on

`configs/particle_lenia.toml` and expect `SystemExit(2)` when no device is present (mark them to skip when a device *is* present).

### R-04 — MaCE input-shape ambiguity

- **Why:** `ece.mace._transport_layout` reads any rank-3 array as `(channels, H, W)`. A single-channel 3D field passed without a channel axis silently gets 2D transport, while the docstrings say `(*spatial)` is accepted.
- **Files:** `ece/mace.py`, `tests/test_mace.py`.
- **Do:** require the channel axis everywhere (rank 3 means 2D with channels, rank 4 means 3D with channels) and update the docstrings, or add an explicit `spatial_dims` argument. Prefer the first; it is the smaller change.
- **Done when:** the docstrings match behavior and a test shows rank-2 input is rejected with a clear message.
- **Verify:** `python3 -m pytest tests/test_mace.py tests/test_field_life.py -q`.

### R-05 — Canonical Orbium for Lenia (M3 exit criterion)

- **Why:** M3's exit is "Orbium persists or dies for documented reasons". Today `configs/lenia_orbium.toml` uses an approximate annular seed that goes extinct by frame 100, and `ece/lenia.py` uses a Gaussian ring kernel (peak 0.5, width 0.15, radius `min(dims)//8`), not the kernel the published Orbium was found with.
- **Files:** `ece/lenia.py`, `configs/lenia_orbium.toml`, `tests/test_lenia.py`, the CUDA/WGSL Lenia modules if the kernel changes, `docs/02-rules-catalog.md`.
- **Do:** implement the kernel and seed from the published Lenia reference (Bert Wang-Chak Chan's Lenia papers and code), including the Orbium cell pattern and its radius. **Obtain the pattern and parameters from that source and cite it in the config comment; do not reconstruct them from memory.** Keep the existing annular seed as a separate, honestly named config if tests still need it.
- **Done when:** a test shows the Orbium keeps total mass within a stated band and its centre of mass moves over at least 1,000 frames; the README and `docs/02` describe the result. If it still dies, the docs state the measured reason.
- **Verify:** `python3 -m pytest tests/test_lenia.py -q`; owner reruns `tests/test_cuda_lenia.py` and `tests/test_wgpu_lenia.py` if the kernel changed.

### R-06 — State hash and `io.hash_every`

- **Why:** `docs/07-determinism.md` specifies a state hash over frame, integer particle state, field, and matrix, but nothing implements it, and the `io.hash_every` config key is accepted and ignored. Replay work (R-14) needs a hash to compare runs across machines.
- **Files:** new `ece/state_hash.py`, `ece/__main__.py`, `docs/07-determinism.md`, new `tests/test_state_hash.py`.
- **Do:** hash the byte layout given in `docs/07` with

`hashlib.blake2b` (standard library; record the algorithm choice in `docs/07`, which currently says xxHash3 or BLAKE3). Start with the fixed-point Mohr state, which is already integer. When `io.hash_every = N > 0`, the CLI prints `frame=<f> hash=<hex>` every N frames.

- **Done when:** identical states hash identically, any one-unit change in any field changes the hash, the layout is little-endian and documented, and the CLI emits hashes on schedule.

- **Verify:** `python3 -m pytest tests/test_state_hash.py tests/test_fixedpoint_mohr.py -q`.

### R-07 — PPS rule (CPU reference)

- **Why:** `"pps"` is an accepted rule name in the loader and schema, but no runner exists; a PPS config loads and then the CLI refuses it.

- **Files:** new `ece/pps.py`, `ece/config.py`, `ece/__main__.py`, `schemas/sim.schema.json`, new `configs/pps.toml`, new `tests/test_pps.py`, `docs/02-rules-catalog.md`.

- **Do:** implement Schmickl's primordial particle system per `docs/02` ($\Delta\phi = \alpha + \beta \cdot N \cdot \text{sign}(R - L)$, constant speed, periodic 2D world), reusing `ece.bins` for neighbour counts. Add a `[pps]` config table (radius, alpha, beta, speed, particles) to the loader key sets and the schema together (R-02's drift test will enforce this).

- **Done when:** a binned-vs-dense agreement test and a determinism test pass, the CLI runs the config and writes NPZ, and `docs/02` points at the implementation.

- **Alternative:** if PPS is not wanted, remove `"pps"` from the rules set and schema instead. Ask the owner which.

### R-08 — Two-way hybrid coupling

- **Why:** the hybrid (`ece/hybrid.py`, `ece/cuda_hybrid.py`) rebuilds the field from particle deposits every frame, applies one MaCE step, and exports `sampled` values, but the field never acts on particle motion. It is a diagnostic, not a coupled system.

- **Decision needed:** the coupling law. Before writing code, ask the owner to choose. Candidates: 1. Gradient drive: `a_i += g · Σ_c W[c_i, c] ∇ρ_c(x_i)`, with a new species-by-channel matrix `W` and gain `g`. 2. Affinity drive: particles climb the MaCE affinity of their own channel, `a_i += g ∇A_{c_i}(x_i)`, reusing the Field Life matrix. Also decide whether the field persists across frames (deposit adds to an evolving ρ) or is rebuilt each frame as now.

- **Files:** `ece/hybrid.py`, `ece/pic.py` (gradient sampling), `ece/config.py` + schema (new keys), `configs/hybrid_pic.toml`, `tests/test_hybrid.py`; GPU versions in follow-up cards.

- **Do (CPU first):** implement the chosen law in the CPU hybrid only, with the gain defaulting to 0 so existing behavior and tests are unchanged. Add the finite-difference test pattern used in `tests/test_particle_lenia.py` for the sampled gradient.

- **Done when:** with gain 0 the hybrid is bit-identical to today; with gain > 0 a test shows particles respond to a fixed synthetic field in the expected direction; docs no longer say "one way".

- **Then:** open separate cards for CUDA and WebGPU parity.

### R-09 — Remove dead CUDA hash kernels

- **Why:** `clear_counts`, `count_bins`, `scan_bins` (single-threaded serial scan), and `scatter_bins` in `ece/cuda_mohr.py` are compiled but never launched; resident planning uses CuPy `argsort`/`bincount`.
- **Files:** `ece/cuda_mohr.py`, `ece/cuda_hybrid.py` (both unpack the kernel tuple).
- **Do:** return only `force_bins` and `integrate` and fix the unpacking.
- **Done when:** no unused kernels remain; CPU suite passes.
- **Verify (owner, cuda):** `python3 -m pytest tests/test_cuda_mohr.py tests/test_cuda_hybrid.py -q`.

### R-10 — CuPy 3D index-overflow guard

- **Why:** the raw kernels in `ece/cupy_field_life.py` index with 32-bit `int`. `channels × depth × height × width ≥ 2³¹` overflows; $512^3 \times 16$ channels is exactly $2^{31}$, and `docs/08` lists $512^3 \times 16$ as a research config. Overflow reads and writes the wrong memory.
- **Files:** `ece/cupy_field_life.py`, `tests/test_cupy_field_life.py`.
- **Do:** either switch kernel indices to `long long` (pass sizes as `np.int64`) or raise a clear `ValueError` before launch when the total reaches $2^{31}$. Prefer the guard if the 64-bit version is slower on GB10.
- **Done when:** oversized inputs fail loudly (or work), and normal sizes are unchanged.
- **Verify:** owner runs `tests/test_cupy_field_life.py` and `python3 tools/benchmark_cupy_field_life.py --sizes 64 128 --frames 3` and confirms timings did not regress.

### R-11 — WebGPU parameter block and GPU hash

- **Why:** every WGSL module passes counts and sizes through an `array<f32>` params buffer, so integers above $2^{2}$■ (~16.7 M) lose precision. The binned WebGPU Mohr path also rebuilds hash tables on the CPU every frame and uploads them.
- **Files:** `ece/wgpu_mohr.py`, `ece/wgpu_field_life.py`, `ece/wgpu_lenia.py`, `ece/wgpu_particle_lenia.py`, `ece/wgpu_pic.py`, `ece/wgpu_preview.py`.
- **Do:** (a) replace the f32 params array with a uniform struct holding `u32` counts and `f32` physical values; (b) as a second change, build count/scan/scatter hash tables in WGSL following `docs/05` (frozen counts, exclusive scan, separate cursor). Keep (a) and (b) in separate PRs.
- **Done when:** all `tests/test_wgpu_*.py` pass on an adapter; for (b), the 200-frame WGSL-vs-CPU Mohr parity test still passes.

### R-12 — CUDA Mohr throughput (FP64 promotion)

- **Why:** the Numba kernels mix float32 arrays with Python float

literals (`0.0`, `0.5`, `1.0`), which promotes the arithmetic to FP64. FP64 throughput on GB10-class hardware is low; this is the likeliest cause of the 1M-particle rate (~5–9 steps/s). It is a hypothesis, not a measurement.

- **Files:** `ece/cuda_mohr.py`, `ece/cuda_hybrid.py`, `ece/cuda_field_life.py`, `ece/cuda_lenia.py`, `ece/cuda_particle_lenia.py`, `ece/cuda_pic.py`.
- **Do:** first measure: record `tools/benchmark_cuda_mohr.py` numbers and inspect the kernels' typed IR (Numba `inspect_types`) for `float64`. Then make constants and accumulators `float32`. Do not enable `fastmath` in the same change; it alters results and needs its own parity check.
- **Done when:** before/after benchmark numbers are recorded in the M5 list with the command and hardware, and all CUDA parity tests pass at their current tolerances.
- **Verify (owner, gb10):** `python3 -m pytest tests/test_cuda_*.py -q; python3 tools/benchmark_cuda_mohr.py --particles 4096 65536 1048576 --frames 10 --constant-density`.

### R-13 — Vectorize CPU fixed-point Mohr

- **Why:** `ece/fixedpoint_mohr.py` is a pure-Python O($N^2$) double loop, about 13 s per frame at 4,096 particles.
- **Files:** `ece/fixedpoint_mohr.py`, `tests/test_fixedpoint_mohr.py`.
- **Do:** vectorize with `numpy.int64`, keeping every rounding step identical (`_round_div` is round-half-away-from-zero). Check headroom: `gain(Q16) × force(Q15) × unit(Q15)` is 2■■ per pair, so a sum over more than about 2¹■ neighbours overflows int64; bin the pairs or keep Python ints for the accumulation.
- **Done when:** the new implementation is bit-identical to the current one on the existing tests plus a 512-particle, 50-frame comparison, and is at least 10× faster.

### R-14 — CUDA fixed-point replay port

- **Why:** cross-machine replay (`docs/07`) needs integer kernels on CUDA with versioned, host-generated lookup tables instead of transcendental functions.
- **Depends on:** R-06 (state hash to compare runs).
- **Files:** new `ece/cuda_fixedpoint_mohr.py`, tests, `docs/07`.
- **Do:** port `ece/fixedpoint_mohr.py` integer-for-integer, including neighbour traversal order. Only friction uses `exp`, and it is computed once on the host; keep it that way.
- **Done when:** the CUDA port produces state hashes identical to the CPU prototype for at least 100 frames on the GB10, and on a second GPU model if one is available.

### R-15 — Long $512^3$ run and memory accounting

- **Why:** the $512^3$ evidence is a ten-frame run (~1.15 s/frame, ~18.8 GB CuPy pool). There is no long-run stability or system-memory number.
- **Depends on:** R-10.
- **Do (gb10):** run $512^3$ × 4 channels for at least 1,000 frames;

record per-channel mass drift, peak CuPy pool, and peak system memory (unified memory). Add a `--report-memory` option to `tools/benchmark_cupy_field_life.py` if needed.

- **Done when:** numbers are recorded below with the exact command; the same for the sustained 256³ default.

### R-16 — WebGPU tests in CI on a software adapter

- **Why:** CI installs no GPU extras, so all `tests/test_wgpu_*.py` skip. A software Vulkan driver (Mesa lavapipe) may let `wgpu` find an adapter on a GitHub runner, which would put the WGSL-vs-CPU parity tests under CI. Unverified; this card is to find out.
- **Files:** `.github/workflows/ci.yml` (a separate, optional job).
- **Do:** in a new job, install `mesa-vulkan-drivers` and `pip install wgpu` (not PySide6), then run `python -m pytest tests/test_wgpu_*.py -q -rs`. Exclude `tests/test_wgpu_preview.py`, which needs a display.
- **Done when:** the job reports the WebGPU compute tests as run (not skipped) and passing; or, if no adapter appears, the card records that finding and is closed. Mark the job `continue-on-error` until it has been green for a few PRs.

### R-17 — Remaining config value validation

- **Why:** the loader is not a complete schema validator. The review confirmed `simulation.dt = nan`, non-finite world lengths, and a one-element `species.range` pass initial checks (the short range then raises `IndexError`). Other numeric table fields are coerced or only checked by the selected runner. Invalid values should fail at load.
- **Files:** `ece/config.py`, `tests/test_config.py`, `schemas/sim.schema.json`, `docs/11-config-schema.md`.
- **Do:** audit remaining schema numeric/array constraints; reject non-finite physical values, invalid types, malformed ranges and dimensions with field-qualified `ValueError`. Keep valid configs and seeded outputs exact. Do not add a runtime `jsonschema` dependency.
- **Done when:** invalid values fail before matrix allocation/execution, shipped configs load, and equivalent valid runs are byte-identical.
- **Verify:** `python3 -m pytest tests/test_config.py -q` and the CPU suite.

## Completed milestones

### M0 — corrected kernels

- [x] Schema/loader drift guard, strict integer seed and species count, ring-only field kernel (R-02); 31 config tests pass on CPU / Python 3.12
- [x] Corrected Mohr tent (no extra `r_max`)
- [x] Exponential friction
- [x] Two-pass MaCE + torus wrap
- [x] Snapshot hash tables
- [x] Invariant tests, document tree, configs, schema, WGSL fragments

- [x] CPU CI on GitHub Actions: Python 3.11 and 3.12, full suite with GPU tests skipping, module compile check, every shipped config loads (R-01)

### M1 — 2D Mohr preview

- [x] Headless CPU frame runner calling `ece.step_mohr` (vectorized binned forces)
- [x] Six-species binned-vs-dense comparison over 200 frames
- [x] Interactive Tk preview with live K×K matrix editor
- [x] CPU, CUDA, and WGSL force backends selectable in the preview
- [x] 200-frame WGSL-vs-CPU per-step oracle comparison on 256 particles
- [x] Native Qt/WebGPU presentation surface with direct screen rendering
- [x] Device-resident native Mohr compute/integrate/render loop (dense $O(N^2)$)
- [x] Hash grid validated against the neighbourhood walk on every backend

Exit (met): a 6-species soup that matches the Python oracle on $N \le 256$ for a few hundred steps within f32 tolerance.

### M2 — Field Life 2D

- [x] Config-driven CPU runner using convolution + MaCE passes
- [x] Per-channel mass-invariance test and NPZ output
- [x] CUDA ring convolution, MaCE passes, and per-channel readback test
- [x] WGSL MaCE passes, CPU parity, and headless CLI output
- [x] Native WebGPU field presentation for Field Life
- [x] CuPy/cuFFT 3D Field Life path with CPU parity at $16^3$ and $256^3$ smoke run

Exit (met): per-channel mass holds; creatures do not evaporate.

### M3 — Lenia growth on the same field buffers

- [x] CPU growth-bell update and normalized radial ring kernel
- [x] Deterministic annular seed and `lenia_orbium.toml` runner; current seed goes extinct by frame 100
- [x] CUDA ring convolution/growth and CPU parity test
- [x] WGSL ring convolution/growth and CPU parity test
- [x] Native WebGPU field presentation for Lenia

Exit (**not met**, see R-05): Orbium persists or dies for documented reasons, not because of a broken clip.

### M4 — PIC coupling

- [x] CPU periodic bilinear particle deposit and field sample references
- [x] Per-species mass and deposit/sample adjoint tests
- [x] Deterministic CUDA segmented deposit and bilinear sample passes
- [x] Deterministic WGSL segmented deposit and bilinear sample passes
- [x] CPU/CUDA/WebGPU hybrid scheduler: Mohr → deposit → Field Life → sample (one-way; see R-08)
- [x] CUDA hybrid keeps particle/velocity/field state and stable hash/PIC planning on-device
- [x] Same-device repeated CUDA hybrid runs produce identical state
- [x] 1024-particle/$128^2$ CUDA hybrid baseline: ~746 steps/s over 100 frames (GB10)
- [x] Single-species CPU Particle Lenia energy-gradient rule and periodic config
- [x] CUDA and WGSL Particle Lenia kernels with CPU oracle tests
- [x] Native Particle Lenia visualization
- [x] Particle Lenia kept as a standalone direct-KDE rule; it does not use PIC deposit/sample

### M5 — CUDA path on Spark (partial; open items are R-10 to R-15)

- [x] Optional CUDA Mohr force kernel (CPU-built hash tables) with oracle comparison

- [x] Device-side hash planning (CuPy stable sort + bincount + scan) and persistent device-resident Mohr integration
- [x] Reproducible stable-sort CUDA benchmark and 4096-particle baseline (GB10: ~600 steps/s, 100 frames)
- [x] Same-device deterministic standalone and hybrid CUDA repeatability tests
- [x] Stable-sort CUDA Mohr scaling probes: 16K at 135 steps/s; 65K at 10.6 steps/s
- [x] Constant-density 1M-particle CUDA run: 10 frames, ~9.0 steps/s, finite torus state; ~5.4 steps/s over 100 frames
- [x] cuFFT-backed 3D field update prototype
- [x] CPU parity and per-channel mass at $16^3$; $256^3$ and $512^3$ feasibility probes
- [x] Fused four-channel cuFFT MaCE benchmark at $64^3$/$128^3$/$256^3$ (2.6 ms/24 ms/0.21 s per frame; ~1.2e-8 mass drift)
- [x] Fused $512^3$ four-channel, ten-frame run (1.15 s/frame; 1.05e-8 mass drift; ~18.8 GB CuPy pool)
- [x] CPU integer Mohr replay prototype in Q16.16/Q8.24/Q1.15 with CLI export and bitwise tests

## Explicitly not scheduled

Do not start these, even as "small" extensions; they need an owner decision to enter the roadmap.

- Co-local 7B VLM steering (design only: `docs/12-ai-evaluator-deferred.md`)
- Full egui IDE / DAG node editor (rule plugin surface in `docs/06` is logical only)

- crates.io release matrix
- 64-species food webs
- $1024^3$ volumes
- Bit-identical replay in WGSL floats (`CORRECTIONS.md` §5)

# Mapping onto Vita, Firebreak, Entelechy

These are product boundaries, not a merge plan.

## Vita / Firebreak

Vita is a Conway-line sandbox (discrete Life + canopy / heightmap / JSON export / Firebreak mechanics). That is Life-like + optional continuous height, not Mohr.

| Vita need | ECE piece |
|---|---|
| Mass that does not evaporate | MaCE / Flow-Lenia transport |
| Discrete Life | separate grid update, not the tent |
| Firebreak as a barrier field | extra Eulerian channel sampled by the CA |
| JSON exports | dump ρ and rule params, not particle SoA |

Do not drive Vita's tick with Mohr forces. If a hybrid is wanted, Firebreak is a field that the CA reads, not a species in `A`.

## Entelechy

Entelechy is the visual graph workbench (React Flow ports / handles / wires, JSON lane packets, object registry). ECE rules become graph nodes later; they are not nodes today.

Suggested port types when the graph wraps ECE:

| Port | Buffer |
|---|---|
| `particles` | SoA pos/vel/type |
| `field_c` | one Eulerian channel |
| `potential` | convolution output |
| `matrix` | K×K |
| `kernel` | radial samples |
| `ranges` | hash table |

Edges: `deposit`, `sample`, `potential`, `force`. A lane packet can carry a `SimConfig` TOML blob plus a matrix checksum.

Daily Entelechy JSON packs stay in that repo. This repo stays the sim kernel + spec.

## PlasmaForge / PIC

Deposit / sample is the same scatter–gather shape as charge deposit in a PIC plasma step. Share the pass idea, not the binary. Plasma needs Poisson / Maxwell; ALife does not.

## SUBTERRA / game work

Game ticks can call M1 Mohr as a toy ecosystem layer. They must not take a dependency on MaCE 3D or the evaluator.

# Config schema

Canonical JSON Schema: `schemas/sim.schema.json`. Human configs: `configs/*.toml`.

## Required block

```
[simulation]
dt = 0.01
world = [1.0, 1.0]          # 2D torus
seed = 1

[species]
count = 6
matrix = "random"           # or "identity", "chase", "path"
range = [-1.0, 1.0]

[mohr]
particles = 4096
r_max = 0.08
beta = 0.3
gain = 1.0
lambda = 6.321             # -ln(0.9)*60 ≈ Mohr gamma 0.9 @ 60 Hz
```

## Optional blocks

```
[field]
dims = [256, 256]
channels = 6
strength = 1.0
crowding_lambda = 0.1
transport_beta = 4.0
kernel = "ring"

[lenia]
mu = 0.15
sigma = 0.015
dt = 0.1

[hash]
cell = 0.08                 # >= r_max
grid = [12, 12]             # world / grid >= r_max; >= 3 bins per axis
neighborhood = 3            # or 5 with cell >= r_max / 2

[io]
headless = true
frames = 1000
hash_every = 0
```

Unknown keys are errors, at the top level and inside every table. A misspelled `lamda` is rejected rather than silently running undamped. The schema uses `"additionalProperties": false` on each object, and `ece/config.py` mirrors those key sets. `tests/test_config.py` checks every table's keys, the rules and field kernel enums, and the seed/species-count bounds against the schema. This is a targeted drift guard, not a complete JSON Schema validator; runtime validation stays dependency-free.

## Matrix modes

| Mode | Fill |
|---|---|
| `random` | U(range), seed from `simulation.seed` |
| `identity` | +1 diagonal, small negative off-diagonal |
| `chase` | cyclic +attract / -flee |
| `path` | load `matrix_path` as row-major f32 text |

## Validation rules

- `simulation.seed` is a non-negative TOML integer; floats, strings and booleans are rejected rather than coerced
- `species.count` is a TOML integer in `[1, 128]`, checked before matrix allocation
- `field.kernel` may only be `"ring"`; `"gaussian"` and `"disk"` are not implemented and are rejected when loading, even in inactive tables
- `beta ∈ (0,1)`
- `r_max > 0`
- `lambda ≥ 0`
- `dt > 0`
- `species.count == matrix side` if a file is loaded
- `hash.neighborhood ∈ {3, 5}`
- `hash.cell >= r_max` for neighborhood 3, `>= r_max / 2` for 5
- `hash.grid` required for a 2D hash; `world / grid >= r_max / (neighborhood // 2)`
- `hash.grid >= neighborhood` bins per axis — a smaller grid wraps the walk onto the same bin twice and double-counts its particles

# Co-local AI evaluator — design only

Not scheduled. Written down so the idea does not get rebuilt from memory when M5 is actually done.

## Why it waits

A VLM cannot debug a wrong tent, a leaking MaCE gather, or a scrambled hash table. Hand-edit the matrix until you can name a creature without a model.

## Contract (when built)

Local multimodal model only. No network.

```
generate_multimodal(system, user, image, opts) -> text
last_embedding() -> optional [f32]
```

Stub backend returns score 0 and no deltas so the sim runs without a model.

## Prompt shape

System: ALife critic. Score 0–1 against a target. Propose sparse `(i,j,δ)` with `δ ∈ [-0.1, 0.1]`. JSON only.

User: target sentence, a few numeric stats (mean speed, spatial entropy, cluster count), $256^2$ thumbnail.

## Steering

```
v ← β v + (1-β) δ
M ← clip(M + η v, -1, 1)
```

Default `η=0.5`, `β=0.7`. Saturation decay if a cell is pushed the same way for N evals. Reject updates that collapse `std(M)` below 0.1.

## Targets that work

Visible structure: “ring of chasers”, “stable bilayer”, “hollow shell with a core”.

Targets that do not work: periods and counts the still frame cannot show (“divides every 100 frames”).

## Cadence

Default every 300 frames. Adaptive: double interval on plateau, halve on a jump. Evaluator thread separate from the GPU tick.

## Memory on Spark (if enabled later)

7B Q4 ~5 GB plus KV cache. Leaves the $256^3$ sim untouched. Full frame readback is for offline analysis, not the live loop.

# File index

| Path | What it is |
|---|---|
| `README.md` | Entry point |
| `AGENTS.md` | Conventions, invariants, and definition of done for coding agents |
| `CLAUDE.md` | Claude Code entry point; imports `AGENTS.md` |
| `.github/workflows/ci.yml` | CPU test suite on every pull request |
| `.gitattributes` | Treat generated PDFs as binary in Git diffs |
| `CORRECTIONS.md` | Deltas vs the original catalog |
| `CHANGELOG.md` | Dated notes |
| `pyproject.toml` | Package metadata |
| `pytest.ini` | Test path |
| `justfile` | Local commands |
| `ece/__init__.py` | Public surface |
| `ece/__main__.py` | Headless CLI runner (`python3 -m ece`) |
| `ece/mohr.py` | Tent + O(N²) accelerations |
| `ece/integrate.py` | Friction + Euler + wrap |
| `ece/mace.py` | Affinity + two-pass MaCE, ring kernels, periodic FFT convolution |
| `ece/bins.py` | Snapshot hash + grid/neighborhood validation |
| `ece/config.py` | TOML loader + validation |
| `ece/matrix.py` | Matrix generators + editor-text parser |
| `ece/step_mohr.py` | CPU Mohr stepper (M1 host), vectorized binned forces |
| `ece/fixedpoint_mohr.py` | CPU integer replay prototype |
| `ece/field_life.py` | CPU Field Life runner |
| `ece/lenia.py` | CPU Lenia runner |
| `ece/particle_lenia.py` | CPU Particle Lenia energy-gradient rule |
| `ece/pic.py` | CPU deposit / sample |
| `ece/hybrid.py` | CPU/CUDA/WebGPU hybrid scheduler |
| `ece/cuda_*.py` | Numba CUDA backends (Mohr, Field Life, Lenia, Particle Lenia, PIC, resident hybrid) |
| `ece/cupy_field_life.py` | CuPy/cuFFT 3D Field Life |
| `ece/wgpu_*.py` | WebGPU compute backends with inline WGSL |
| `ece/preview.py` | Tk preview with matrix editor |
| `ece/wgpu_preview.py` | Native Qt/WebGPU preview |
| `shaders/mohr_force.wgsl` | Corrected force reference fragment |
| `shaders/mace_2pass.wgsl` | Denom + gather reference fragment |
| `tests/test_*.py` | Invariants and backend oracle tests; see `docs/15-testing.md` |
| `tools/benchmark_*.py` | CUDA Mohr and CuPy Field Life benchmarks |
| `configs/*.toml` | Named soups |

| Path | What it is |
|---|---|
| `schemas/sim.schema.json` | Config JSON Schema |
| `reports/REVIEW_2026-10-04_R02.md` | Published-source review and schema/loader regression evidence |
| `docs/00–15` | Design set |
| `docs/handbook.pdf` | Printable compilation |
| `tools/build_handbook.py` | Handbook builder |

# Shader fragments

These are math-correct reference snippets, not a full wgpu project, and no Python module loads them. The executable WGSL lives inline in `ece/wgpu_*.py` (Mohr, Field Life, Lenia, Particle Lenia, PIC, and the native preview).

`shaders/mohr_force.wgsl`

- `mohr_force(r, a, beta)` — dimensionless tent
- `wrapped_delta_2d` — minimum image
- Force loop body multiplies by `params.gain` only
- Integrate uses `exp(-lambda * dt)`

`shaders/mace_2pass.wgsl`

- `mace_denom` writes `Z`
- `mace_gather` reads `Z` and wraps with `(x % n + n) % n`
- One channel shown; dispatch once per channel or flatten `CXY`

## Bindings the host must supply

Mohr force pass: `SimParams`, positions, sorted indices, ranges, matrix, accel out.

MaCE: `SimParams` with `field_w/h` and `transport_beta`, `affinity`, `rho_in`, `denom`, `rho_out`.

## Not provided

Prefix-sum shaders, FFT, deposit CAS, render. The `ece/wgpu_*.py` hosts cover deposit (segmented, no CAS) and render; hash tables are still built on the CPU.

# Testing

```
python3 -m pytest tests/ -q
```

118 tests after the M1 preview, field runners, PIC references, replay tests, and hash/config guards. Without a GPU, 78 run and 40 skip; no test needs `tkinter`.

| File | Locks |
|---|---|
| `test_mohr.py` | wall, tent peak, horizon, scale invariance, asymmetry, wrap |
| `test_mace.py` | per-channel mass, uniform fixed point, corner torus, local-max softmax, crowding |
| `test_bins_integrate.py` | exclusive scan, frozen snapshot, torus-aligned bins, grid ≥ neighborhood, Mohr-gamma conversion, wrap |
| `test_config.py` | TOML load, chase matrix, channel mismatch, invalid hash grid, unknown top-level and in-table keys, schema drift, strict integer seed/count bounds, unsupported field kernels, all shipped configs |
| `test_hybrid.py` | CPU hybrid scheduling, per-species mass, sampled values, NPZ output |
| `test_cuda_mohr.py` | optional CUDA-vs-CPU step, resident stability, CLI smoke, same-device repeatability (skips without device) |
| `test_cuda_field_life.py` | optional CUDA-vs-CPU three-step field parity, mass readback, and CLI NPZ output |
| `test_cuda_hybrid.py` | optional CUDA hybrid parity, mass readback, 100-frame stability, CLI, and same-device repeatability |
| `test_cuda_lenia.py` | optional CUDA-vs-CPU growth, boundedness, and CLI NPZ output |
| `test_cuda_particle_lenia.py` | optional CUDA-vs-CPU energy gradient, wrap stability, and CLI output |
| `test_cuda_pic.py` | optional CUDA segmented deposit and sample parity (skips without device) |
| `test_wgpu_mohr.py` | optional WGSL-vs-CPU 200-frame parity at 256 particles, CLI smoke, and stability (skips without adapter) |
| `test_wgpu_field_life.py` | optional WGSL-vs-CPU three-step field parity, mass readback, and CLI NPZ output |
| `test_wgpu_lenia.py` | optional WGSL-vs-CPU growth, bounds, and CLI NPZ output |
| `test_wgpu_particle_lenia.py` | optional WGSL-vs-CPU energy gradient, wrap stability, and CLI output |
| `test_wgpu_pic.py` | optional WGSL segmented deposit/sample parity and mass conservation |
| `test_wgpu_hybrid.py` | optional one-frame hybrid parity and full-state CLI output |
| `test_wgpu_preview.py` | optional native-surface modes plus device-resident Mohr step/reset |
| `test_field_life.py` | config-driven runner, per-channel mass, field NPZ output |
| `test_cupy_field_life.py` | optional 3D CuPy/cuFFT parity, mass conservation, and CLI volume output |
| `test_lenia.py` | kernel normalization, bounded growth, configured extinction, Lenia NPZ output |
| `test_particle_lenia.py` | shell normalization, energy-gradient finite difference, periodic run, CLI state export |
| `test_fixedpoint_mohr.py` | fixed-point oracle tolerance, integer repeatability, wrap bounds, CLI Q-state export |
| `test_pic.py` | per-species deposit mass, torus sampling, deposit/sample adjoint identity |

| File | Locks |
|---|---|
| `test_step_mohr.py` | torus stay-in, per-frame binned vs dense agreement (incl. minimal 3×3 grid and clustered particles), pair-budget chunking, headless CLI, matrix validation |

CI (`.github/workflows/ci.yml`) runs this suite on Python 3.11 and 3.12 for every pull request and push to `main`, without GPU extras, then compiles every module and loads every config in `configs/`.

GPU tests skip cleanly when their adapter is unavailable. Small-N CPU oracle parity and per-channel mass readback are the primary backend gates.

# Corrections applied to the catalog / ECE spec

These replace the corresponding formulas in `particlelifesim.txt`. The Python package `ece/` is the executable source of truth.

## 1. Mohr force — drop the extra `r_max`

**Was (catalog acceleration):**

```
accel += F * r_max * unit(other - me)
```

**Correct:**

```
r = dist / r_max
F = tent(r, a, beta)          # dimensionless
accel += gain * F * unit(other - me)
```

`gain` defaults to `1`. Peak mid-range force is then exactly the matrix entry `a`, independent of world scale. Scaling the world and `r_max` together must not change accelerations (see `test_force_independent_of_rmax_scale`).

Contact wall (`r < beta`) stays species-independent and strictly repulsive. Clusters-style per-pair `(r1,r2,f1,f2)` may be added later; `f1` must not be attractive.

## 2. Friction — explicit time constant

**Was:** `v *= gamma ** (60 * dt)` (assumes 60 Hz)

**Correct:** `v = exp(-lambda dt)`

Conversion if you want Mohr's feel: `lambda = -ln(gamma) * 60`.

## 3. MaCE — two passes, torus wrap, no border clip

**Pass A** — denominators

```
Z[x] = sum_{y in N(x)} exp(beta * A[y])
```

**Pass B** — gather

```
rho'[x] = sum_{y in N(x)} rho[y] * exp(beta * A[x]) / Z[y]
```

`N(x)` is the 9-cell (2D) or 27-cell (3D) neighborhood **including self**, wrapped on the torus. Do not `continue` on out-of-bounds. Clipping leaks mass at the box face.

Invariant: `sum(rho'_c) == sum(rho_c)` per channel, up to float rounding. Momentum is not conserved.

## 4. Spatial hash — freeze the snapshot

Keep four distinct arrays. Never scan a counter you later reuse as a write cursor.

| Buffer | Role |
|---|---|
| `counts` | frozen particle-per-bin snapshot |
| `offsets` | exclusive prefix sum of `counts` |

| Buffer | Role |
|---|---|
| `ranges` | `(offsets[b], offsets[b] + counts[b])` |
| `cursor` | scatter write head, discarded after the pass |

`cell_size >= r_max` if you walk a 3×3 neighborhood.

## 5. Scope cut (do not implement from the catalog yet)

M1 is 2D Mohr + matrix + these tests. Not in M1:

- 3D $512^3$ volumes
- 64-species Field Life
- CAS atomic f32 deposit
- co-local VLM
- full egui IDE / release matrix

wgpu is the interactive preview. Bit-identical replay, if required, lives on a fixed-point CUDA path later — not in WGSL floats.

## 6. DGX Spark budget for the first real run

Default: $256^2$ or $256^3 \times 8$ channels, 1–4 M particles, cuFFT when kernel radius > ~6 in 3D. 128 GB is capacity. GB10-class bandwidth is not an H100. Locality and mixed precision first.

# Changelog

## Unreleased

- Config schema/loader drift guard (R-02): table keys, rule/kernel enums, and seed/species-count bounds are regression-tested together. The loader rejects non-integer/negative seeds, non-integer species counts outside `[1, 128]`, and unsupported field kernels before execution. The schema now advertises only the implemented ring kernel; no new dependency.
- Spatial hash: reject grids with fewer bins per axis than the neighborhood walk (they double-counted wrapped bins); validated by the loader and by the CPU, CUDA, and WebGPU binned force paths.
- Config: unknown keys inside any table are now errors, matching the schema.
- CPU binned Mohr force is vectorized (~19× faster at 4,096 particles), with work chunked by candidate-pair count so clustered particles stay bounded in memory (4,096 particles in one bin: 1.9 GB peak → 0.18 GB).
- `tests/test_step_mohr.py` no longer requires `tkinter`; the matrix parser moved to `ece.matrix`.
- Roadmap rewritten as task cards (R-01 to R-15) with files, acceptance criteria, verification commands, and hardware needs; added `AGENTS.md` (conventions, invariants, definition of done) and `CLAUDE.md`.
- CPU CI on GitHub Actions (R-01): Python 3.11 and 3.12, full test suite without GPU extras, module compile check, and config loading.
- Docs reconciled with the code: status, hybrid one-way coupling, config example, hash rules, file index, shader notes, benchmark figures.

## 0.1.0 — 2026-09-28

- Implemented review corrections as executable kernels.
- Mohr tent: removed extra `r_max` scale; added `gain`.
- Friction: `exp(-λ dt)` with Mohr-gamma conversion helper.
- MaCE: two-pass denom + gather, torus wrap, per-channel mass tests.
- Spatial hash: frozen snapshot, exclusive scan, separate cursor.
- Document set `docs/00`–`13`, JSON schema, five TOML configs.
- CPU Mohr stepper with binned vs dense agreement test.
- WGSL fragments matching the corrected math.
- Printable handbook builder.